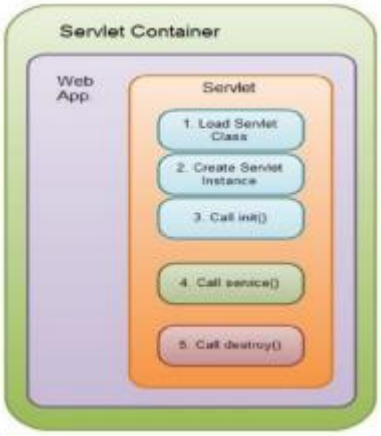
1. **What is a Servlet? Explain the lifecycle of a servlet?**

A. Servlets are programs that run on a Web or Application server act as a middle layer between a request coming from a Web browser or other HTTP client and databases or applications on the HTTP server.
Using Servlets, you can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically.
Servlets are server side components that provide a powerful mechanism for developing web applications.

A servlet life cycle can be defined as the entire process from its creation till the destruction. The following are the paths followed by a servlet
The servlet is initialized by calling the init () method.
The servlet calls service() method to process a client's request.
The servlet is terminated by calling the destroy() method.
Finally, servlet is garbage collected by the garbage collector of the JVM.

S.



The init() method :

- The init method is designed to be called only once.
- It is called when the servlet is first created, and not called again for each user request. So, it is used for one-time initializations, just as with the init method of applets.
- The servlet is normally created when a user first invokes a URL corresponding to the servlet, but you can also specify that the servlet be loaded when the server is first started.
- The init() method simply creates or loads some data that will be used throughout the life of the servlet.

The init method definition looks like this:

```
public void init() throws ServletException {
// Initialization code...
}
```

The service() method :

- The service() method is the main method to perform the actual task.

- The servlet container (i.e. web server) calls the service() method to handle requests coming from the client(browsers) and to write the formatted response back to the client.
- Each time the server receives a request for a servlet, the server spawns a new thread and calls service.
- The service() method checks the HTTP request type (GET, POST, PUT, DELETE, etc.) and calls doGet, doPost, doPut, doDelete, etc. methods as appropriate.

Signature of service method:

```
public void service(ServletRequest request, ServletResponse response)
throws ServletException, IOException
{
}
```

The service () method is called by the container and service method invokes doGe, doPost, doPut, doDelete, etc.methods as appropriate. So you have nothing to do with service() method but you override either doGet() or doPost() depending on what type of request you receive from the client.
The doGet() and doPost() are most frequently used methods with in each service request. Here is the signature of these two methods.

The doGet() Method

A GET request results from a normal request for a URL or from an HTML form that has no METHOD specified and it should be handled by doGet() method.

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
// Servlet code
}
```

The doPost() Method

A POST request results from an HTML form that specifically lists POST as the METHOD and it should be handled by doPost() method.

```
public void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException
{
// Servlet code
}
```

The destroy() method :

The destroy() method is called only once at the end of the life cycle of a servlet.
This method gives your servlet a chance to close database connections, halt background threads, write cookie lists or hit counts to disk, and perform other such cleanup activities.
After the destroy() method is called, the servlet object is marked for garbage collection.
The destroy method definition looks like this:

```
public void destroy() {
// Finalization code...
}
```

2. What is Cookie? Explain creation of Cookie and retrieving information from cookie using code snippets?

Cookies are small bits of textual information that a web server sends to a browser and that the browser later returns unchanged when visiting the same web site or domain

Sending cookies to the client:

1.Creating a cookie object: Create an object for the cookie class
• Cookie():constructs a cookie.
• Cookie(String name, String value)constructs a cookie with a specified name and value.
EX:

```
Cookie ck=new Cookie("user",”mca");
```

2.Setting the maximum age
setMaxAge() is used to specify how long (in seconds) the cookie should be valid.
Ex:cookie.setMaxAge(60*60*24);
3.Placing the cookie into the HTTP response headers.
We use response.addCookie to add cookies in the HTTP response header as follows:
response.addCookie(cookie);
Reading cookies from the client:
1. Call request.getCookies(). This yields an array of cookie objects.
2. Loop down the array, calling getName on each one until you find the cookie of interest.
Ex:

```
String cookieName=“userID”;
Cookie[] cookies=request.getCookies();
If(cookies!=null)
{
for(int i=0;i<cookies.length;i++){
Cookie cookie=cookies[i];
if(cookieName.equals(cookie.getName())){
doSomethingwith(cookie.getValue());
}}}
```

3. With an example, describe the various tags available in JSP.

JSP scriptlet tag:A scriptlet tag java source code in JSP.

<% java source code %>

In this example, we are displaying a welcome message.

<html>

<body>

<% out.print(“Welcome to jsp” %>

</body>

</html>

JSP Declaration Tag: The JSP declaration tag is used to declare variables, objects and methods.

The code written inside the jsp declaration tag is placed outside the service() method of auto

generated servlet. So it doesn't get memory at each request.

<%! Field or method declaration %>

Ex:

<html>

<body>

<%! Int data=50; %>

<%= “value “ +data %>

</body>

</html>

JSP Expression Tag:

Expression Tag is used to print out java language expression that is put between the tags. An expression tag can hold any java language expression that can be used as an argument to the out.print() method.

Syntax :<%= java expression %>

<%= (2*5) %>

JSP Directives:

The jsp directives are messages that tells the web container how to translate a JSP page into the corresponding servlet.

Syntax

<%@ directive attribute=”value” %>

There are three types of directives: 1. import 2. include directive directive 3. taglib directive

4) What is JSP? Explain the advantages of JSP over Servlet.

- It is Extension to Servlet Technology.
- It has all the features of servlet and additionally has implicit objects, predefined tags, custom tags
- It is easily managed. It separates business logic with presentation logic

- It can be easily deployed
- If JSP page is modified, no need to redeploy but if changes are required in servlet , the entire code needs to be updated and recompile

JSP	Servlet
JSP is a web page scripting language that can generate dynamic content	**Servlets are Java programs that already compiled which also creates dynamic web content**
JSP runs slower compared to servlet as it takes compilation time to convert into servlets	**Servlets run faster compared to JSP.**
It's easier to code in JSP than in Java	**It is not easier when compared to JSP**
In MVC, jsp act as a view.	**In MVC, servlet act as a controller.**
JSP are generally preferred when there is not much processing of data required.	**servlets are best for use when there is more processing and manipulation involved.**
The advantage of JSP programming over servlets is that we can build custom tags which can directly call Java beans.	**There is no such custom tag facility in servlets.**
We can achieve functionality of JSP at client side by running JavaScript at client side.	**There are no such methods for servlets.**

5) What are the different implicit objects in JSP? Explain with an example.

JSP out implicit object

For writing any data to the buffer, JSP provides an implicit object named out.

It is the object of JspWriter.

- In case of servlet you need to write:

```
PrintWriter out=response.getWriter();
```

- But in JSP, you don't need to write this code.

E**xample of out implicit object**

- <html>
- <body>
- <% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
- </body>
- </html>

Request :

- The JSP request is an implicit object of type HttpServletRequest i.e. created for each jsp request by the web container.
- It can be used to get request information such as parameter, header information, remote address, server name, server port, content type, character encoding etc.
- It can also be used to set, get and remove attributes from the jsp request scope.

index.html

- <form action="welcome.jsp">
- <input type="text" name="uname">
- <input type="submit" value="go">
- </form>
- welcome.jsp
- <%
- String name=request.getParameter("uname");
- out.print("welcome "+name); %>

Response Object :

- In JSP, response is an implicit object of type HttpServletResponse. The instance of HttpServletResponse is created by the web container for each jsp request.
- It can be used to add or manipulate response such as redirect response to another resource, send error etc.

index.html

- **<form** action="welcome.jsp">
- **<input** type="text" name="uname">
- **<input** type="submit" value="go"><**br/**>
- **</form>**
- **welcome.jsp**
- <%
- response.sendRedirect("http://www.google.com");
- %>

Config:

- In JSP, config is an implicit object of type *ServletConfig*. This object can be used to get initialization parameter for a particular JSP page. The config object is created by the web container for each jsp page.
- Generally, it is used to get initialization parameter from the web.xml file.
- **index.html**

```
<form action="welcome">

<input type="text" name="uname">

<input type="submit" value="go"><br/>

</form>
```

- **web.xml file**

```
<web-app>

<servlet>

<servlet-name>sonoojaiswal</servlet-name>

<jsp-file>/welcome.jsp</jsp-file>

<init-param>

<param-name>dname</param-name>

<param-value>sun.jdbc.odbc.JdbcOdbcDriver</param-value>

</init-param>

</servlet>

<servlet-mapping>

<servlet-name>sonoojaiswal</servlet-name>

<url-pattern>/welcome</url-pattern>

</servlet-mapping>

</web-app>
```

welcome.jsp

- <%
- out.print("Welcome "+request.getParameter("uname"));
-
- String driver=config.getInitParameter("dname");
- out.print("driver name is="+driver);
- %>

Application:

- In JSP, application is an implicit object of type *ServletContext*.
- The instance of ServletContext is created only once by the web container when application or project is deployed on the server.
- This object can be used to get initialization parameter from configuaration file (web.xml). It can also be used to get, set or remove attribute from the application scope.

This initialization parameter can be used by all jsp pages.

Example

- **index.html**

  <form action="welcome"**>**

  input type="text" name="uname"**>**

  <input type="submit" value="go"**>**

  </form>
- **welcome.jsp**

  <%

  out.print("Welcome "+request.getParameter("uname"));

  String driver=application.getInitParameter("dname");

  out.print("driver name is="+driver);

  %>

- **<web-app>**
- **<servlet>**
- **<servlet-name>**sonoojaiswal**</servlet-name>**
- **<jsp-file>**/welcome.jsp**</jsp-file>**
- **</servlet>**
- **<servlet-mapping>**
- **<servlet-name>**sonoojaiswal**</servlet-name>**
- **<url-pattern>**/welcome**</url-pattern>**
- **</servlet-mapping>**
- **<context-param>**
- **<param-name>**dname**</param-name>**
- **<param-value>**sun.jdbc.odbc.JdbcOdbcDriver**</param-value>**
- **</context-param>**
- **</web-app>**

Session:

- In JSP, session is an implicit object of type HttpSession. The Java developer can use this object to set,get or remove attribute or to get session information.
- **index.html**
- <html>
- <body>
- <form action="welcome.jsp">
- <input type="text" name="uname">

- <input type="submit" value="go">
- </form>
- </body>

</html>

Example

- **welcome.jsp**
- <html>
- <body>
- <%
- String name=request.getParameter("uname");
- out.print("Welcome "+name);
- session.setAttribute("user",name);
- second jsp page
- %>
- </body>
- </html>

- **second.jsp**
- <html>
- <body>
- <%
-
- String name=(String)session.getAttribute("user");
- out.print("Hello "+name);
-
- %>
- </body>
- </html>

Pagecontext:

- In JSP, pageContext is an implicit object of type PageContext class.The pageContext object can be used to set,get or remove attribute from one of the following scopes:page
- request
- session
- application
- In JSP, page scope is the default scope.
- **index.html**
- <html>
- <body>
- <form action="welcome.jsp">
- <input type="text" name="uname">
- <input type="submit" value="go">
- </form>
- </body>
- </html>

- **welcome.jsp**
- <html>
- <body>
- <%
-
- String name=request.getParameter("uname");
- out.print("Welcome "+name);
-
- pageContext.setAttribute("user",name,PageContext.SESSION_SCOPE);
-
- second jsp page
-
- %>
- </body>
- </html>

- **second.jsp**
- <html>
- <body>
- <%
-
- String name=(String)pageContext.getAttribute("user",PageContext.SESSION_SCOPE);
- out.print("Hello "+name);
-
- %>
- </body>
- </html>

Page:

- In JSP, page is an implicit object of type Object class.
- This object is assigned to the reference of auto generated servlet class.
- It is written as:Object page=this;
- For using this object it must be cast to Servlet type.
- For example:<% (HttpServlet)page.log("message"); %>
- Since, it is of type Object it is less used because you can use this object directly in jsp.For example:<% this.log("message"); %>

Exception:

- In JSP, exception is an implicit object of type java.lang.Throwable class. This object can be used to print the exception. But it can only be used in error pages.It is better to learn it after page directive. Let's see a simple example:
- **error.jsp**
- <%@ page isErrorPage="true" %>
- <html>
- <body>
-
- Sorry following exception occured:<%= exception %>
-
- </body>
- </html>

6) Explain how do you handle HTTP requests and generate responses.

The HttpServlet class provides specialized methods that handle the various types of HTTP requests. A servlet developer typically overrides one of these methods. These methods are doDelete( ), doGet( ), doHead( ), doOptions( ), doPost( ), doPut( ), and doTrace( ). Acomplete description of the different types of HTTP requests is beyond the scope of this book. However, the GET and POST requests are commonly used when handling form input. Therefore, this section presents examples of these cases.

Handling HTTP GET Requests

The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in ColorGet.htm, and a servlet is defined in ColorGetServlet.java. The HTML source code for ColorGet.htm is shown in the following listing. It defines a form that contains a select element and a submit button. Notice that the action parameter of the form tag specifies a URL.

The URLidentifies a servlet to process the HTTP GET request.

```
<html>
<body>
<center>
<form name="Form1"
action="http://localhost:8080/servlets-examples/servlet/ColorGetServlet">
<B>Color:</B>
<select name="color" size="1">
<option value=”red”>RED</option>
<option value=”blue”>BLUE</option>
```

<option value=”green”>GREEN</option>

</select>

<input type=submit value="Submit">

</form>

</body>

</html>

The source code for ColorGetServlet.java is shown in the following listing. The doGet( ) method is overridden to process any HTTP GET requests that are sent to this servlet. It uses the getParameter( ) method of HttpServletRequest to obtain the selection that was made by the user. A response is then formulated.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorGetServlet extends HttpServlet {
public void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
String color = request.getParameter("color");


response.setContentType("text/html");
PrintWriterpw = response.getWriter();
pw.println("<B>The selected color is: ");
```

```
pw.println(color);
pw.close();
}
}
```

Handling Post requests:

The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in ColorPost.htm, and a servlet is defined in ColorPostServlet.java. The HTMLsourcecode for ColorPost.htm is shown in the following listing. It is identical to ColorGet.htm except that the method parameter for the form tag explicitly specifies that the POST method should be used, and the action parameter for the form tag specifies a different servlet.

```
<html>
<body>
<center>
<form name="Form1"
method="post"
action="http://localhost:8080/servlets-examples/servlet/ColorPostServlet">
<B>Color:</B>
<select name="color" size="1">
<option value="Red">Red</option>
<option value="Green">Green</option>
<option value="Blue">Blue</option>
</select>
```

```
<br><br>
<input type=submit value="Submit">
</form>
</body>
</html>
```

The source code for ColorPostServlet.java is shown in the following listing. The doPost( ) method is overridden to process any HTTP POST requests that are sent to this servlet. It uses the getParameter( ) method of HttpServletRequest to obtain the selection that was made by the user. A response is then formulated.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorPostServlet extends HttpServlet {
public void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
String color = request.getParameter("color");
response.setContentType("text/html");
PrintWriterpw = response.getWriter();
pw.println("<B>The selected color is: ");

pw.println(color);
pw.close();
```

}

}

7. Explain JSTL Library?

- Java Server Pages Standard Tag Library (JSTL) is a collection of useful JSP tags which encapsulates core functionality common to many JSP applications.
- JSTL has support for common, structural tasks such as iteration and conditionals, tags for manipulating XML documents, internationalization tags, and SQL tags. It also provides a framework for integrating existing custom tags with JSTL tags.

Advantage of JSTL

- Fast Developement JSTL provides many tags that simplifies the JSP.
- Code Reusability We can use the JSTL tags in various pages.

No need to use scriptlet tag It avoids the use of scriptlet tagThe JSTL tags can be classified, according to their functions, into following JSTL tag library groups that can be used when creating a JSP page:

- Core Tags
- Formatting tags
- SQL tags
- XML tags
- JSTL Functions

Tag Name	Description
Core tags	The JSTL core tag provide variable support, URL management, flow control etc. The url for the core tag is **http://java.sun.com/jsp/jstl/core** . The prefix of core tag is **c**.
Function tags	The functions tags provide support for string manipulation and string length. The url for the functions tags is **http://java.sun.com/jsp/jstl/functions** and prefix is **fn**.
Formatting tags	The Formatting tags provide support for message formatting, number and date formatting etc. The url for the Formatting tags is **http://java.sun.com/jsp/jstl/fmt** and prefix is **fmt**.
XML tags	The xml sql tags provide flow control, transformation etc. The url for the xml tags is **http://java.sun.com/jsp/jstl/xml**and prefix is **x**.
SQL tags	The JSTL sql tags provide SQL support. The url for the sql tags is **http://java.sun.com/jsp/jstl/sql** and prefix is **sql**.

Core Tags:

- The core group of tags are the most frequently used JSTL tags. Following is the syntax to include JSTL Core library in your JSP:
- <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

Tag	Description
<c:out >	Like <%= ... >, but for expressions.
<c:set >	Sets the result of an expression evaluation in a 'scope'
<c:remove >	Removes a scoped variable (from a particular scope, if specified).
<c:catch>	Catches any Throwable that occurs in its body and optionally exposes it.
<c:if>	Simple conditional tag which evalutes its body if the supplied condition is true.
<c:choose>	Simple conditional tag that establishes a context for mutually exclusive
	conditional operations, marked by <when> and <otherwise>
<c:when>	Subtag of <choose> that includes its body if its condition evaluates to 'true'.
<c:otherwise >	Subtag of <choose> that follows <when> tags and runs only if all of the prior conditions evaluated to 'false'.
<c:import>	Retrieves an absolute or relative URL and exposes its contents to either the page, a String in 'var', or a Reader in 'varReader'.
<c:forEach >	The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
<c:forTokens>	Iterates over tokens, separated by the supplied delimiters.
<c:param>	Adds a parameter to a containing 'import' tag's URL.
<c:redirect >	Redirects to a new URL.
<c:url>	Creates a URL with optional query pa

SQL Tags:

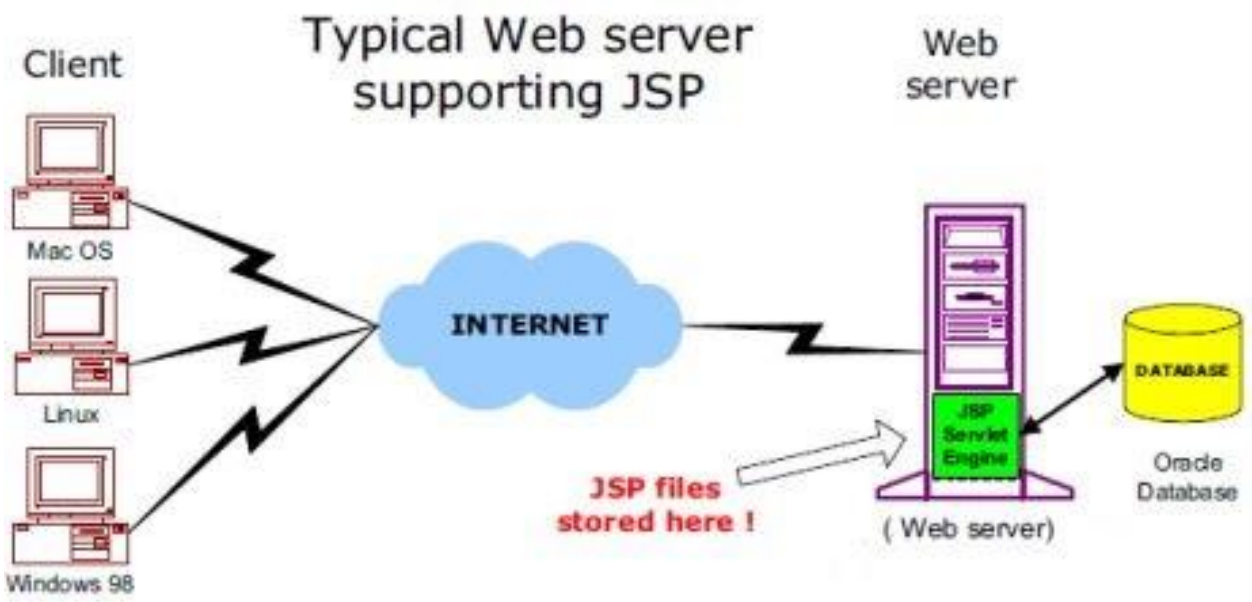
- The JSTL SQL tag library provides tags for interacting with relational databases (RDBMSs) such as Oracle, mySQL, or Microsoft SQL Server.

Following is the syntax to include JSTL SQL library in your JSP:

- <%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>

Tag	Description
<sql:setDataSource>	Creates a simple DataSource suitable only for prototyping
<sql:query>	Executes the SQL query defined in its body or through the sql attribute.
<sql:update>	Executes the SQL update defined in its body or through the sql attribute.
<sql:param>	Sets a parameter in an SQL statement to the specified value.
<sql:dateParam>	Sets a parameter in an SQL statement to the specified java.util.Date value.
<sql:transaction >	Provides nested database action elements with a shared Connection, set up to execute all statements as one transaction.

8. Explain JSP architecture



JSP Processing:

1. Browser sends an HTTP request to the web server.
2. The web server recognizes that the HTTP request is for a JSP page **(.jsp page**) and forwards it to a JSP engine.
3. The JSP engine loads the JSP page from disk and converts it into a servlet content. All template text is converted to println( ) statements and all JSP elements are converted to Java code that implements the corresponding dynamic behavior of the page.
4. The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.

5. Servlet engine loads the Servlet class and executes it. & produces an output in HTML format, which the servlet engine passes to the web server inside an HTTP response.
6. The web server forwards the HTTP response to your browser in terms of static HTML content.

9. Explain the HTTP request Headers

When a browser requests for a web page, it sends lot of information to the web server which can not be read directly because this information travel as a part of header of HTTP request. You can check HTTP Protocol for more information on this.

Following is the important header information which comes from browser side and you would use very frequently in web programming:

Header	Description
Accept	This header specifies the MIME types that the browser or other clients can handle. Values of **image/png** or**image/jpeg** are the two most common possibilities.
Accept-Charset	This header specifies the character sets the browser can use to display the information. For example ISO-8859-1.
Accept-Encoding	This header specifies the types of encodings that the browser knows how to handle. Values of **gzip** or**compress** are the two most common possibilities.
Accept-Language	This header specifies the client's preferred languages in case the servlet can produce results in more than one language. For example en, en-us, ru, etc.
Authorization	This header is used by clients to identify themselves when accessing password-protected Web pages.
Connection	This header indicates whether the client can handle persistent HTTP connections. Persistent connections permit the client or other browser to retrieve multiple files with a single request. A value of **Keep-Alive** means that persistent connections should be used
Content-Length	This header is applicable only to POST requests and gives the size of the POST data in bytes.
Cookie	This header returns cookies to servers that previously sent them to the

	browser.
Host	This header specifies the host and port as given in the original URL.
If-Modified-Since	This header indicates that the client wants the page only if it has been changed after the specified date. The server sends a code, 304 which means **Not Modified**header if no newer result is available.
If-Unmodified-Since	This header is the reverse of If-Modified-Since; it specifies that the operation should succeed only if the document is older than the specified date.
Referer	This header indicates the URL of the referring Web page. For example, if you are at Web page 1 and click on a link to Web page 2, the URL of Web page 1 is included in the Referer header when the browser requests Web page 2.
User-Agent	This header identifies the browser or other client making the request and can be used to return different content to different types of browsers.

Methods to read HTTP Header:

There are following methods which can be used to read HTTP header in your servlet program.

These methods are available with *HttpServletRequest* object.

- **getCookies**

The getCookies method returns the contents of the Cookie header, parsed and stored in an array of Cookie objects.

• getAuthType and getRemoteUser

The getAuthType and getRemoteUser methods break the Authorization header into its component pieces.

• getContentLength

The getContentLength method returns the value of the Content-Length header (as an int).

getContentType

The getContentType method returns the value of the Content-Type header (as a String).

• getDateHeader and getIntHeader

The getDateHeader and getIntHeader methods read the specified header and then convert them to Date and int values, respectively.

• getHeaderNames

Rather than looking up one particular header, you can use the getHeaderNames method to get an Enumeration of all header names received on this particular request.

• getHeaders

In most cases, each header name appears only once in the request. Occasionally, however, a header can appear multiple times, with each occurrence listing a separate value.

• getMethod

The getMethod method returns the main request method (normally GET or POST, but things like HEAD, PUT, and DELETE are possible).

• **getRequestURI**

The getRequestURI method returns the part of the URL that comes after the host and port but before the form data. For example, for a URL of http://randomhost.com/servlet/search.BookSearch, getRequestURI would return /servlet/search.BookSearch.

• **getProtocol**

Lastly, the getProtocol method returns the third part of the request line, which is generally HTTP/1.0 or HTTP/1.1.

Example program to print All Headers

```
package coreservlets;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;
/** Shows all the request headers sent on this particular request.*/
public class ShowRequestHeaders extends HttpServlet
{
        public void doGet(HttpServletRequest request,HttpServletResponse response)
                throws ServletException, IOException
        {
                response.setContentType("text/html");
                PrintWriter out = response.getWriter();
                String title = "Servlet Example: Showing Request Headers";
                out.println("Request Method: " + request.getMethod());
                out.println("Request URI: " + request.getRequestURI());
                out.println("Request Protocol: " + request.getProtocol());
                Enumeration headerNames = request.getHeaderNames();
                while(headerNames.hasMoreElements())
                {
                   String headerName = (String)headerNames.nextElement();
                   out.println(headerName);
                   out.println(" " + request.getHeader(headerName));
                }
}
```

10.Explain the HTTP response Headers

When a Web server responds to a HTTP request to the browser, the response typically consists of a status line, some response headers, a blank line, and the document. A typical response looks like this:

```
HTTP/1.1 200 OK
Content-Type: text/html
Header2: ...
...
HeaderN: ...
 (Blank Line)
<!doctype ...>
<html>
<head>...</head>
<body>
...
</body>
</html>
```

The status line consists of the HTTP version (HTTP/1.1 in the example), a status code (200 in the example), and a very short message corresponding to the status code (OK in the example).

Following is a summary of the most useful HTTP 1.1 response headers which go back to the browser from web server side and you would use them very frequently in web programming:

Header	Description
Allow	This header specifies the request methods (GET, POST, etc.) that the server supports.
Cache-Control	This header specifies the circumstances in which the response document can safely be cached. It can have values **public, private** or **no-cache** etc. Public means document is cacheable, Private means document is for a single user and can only be stored in private (nonshared) caches and no-cache means document should never be cached. • no-store: Document should never be cached and should not even be stored in a temporary location on disk. This header is intended to prevent inadvertent copies of sensitive information. • must-revalidate: Client must revalidate document with original server (not just intermediate proxies) each time it is used. • proxy-revalidate: This is the same as must-revalidate, except that it applies only to shared caches. • max-age=*xxx*: Document should be considered stale after *xxx* seconds. This is a convenient alternative to the Expires header, but only works with HTTP 1.1 clients. If both max-age and Expires are present in the response, the max-age value takes precedence. • s-max-age=*xxx*: Shared caches should consider the document stale after *xxx*

	seconds.
	response.setHeader("Cache-Control", "no-cache");
	response.setHeader("Pragma", "no-cache");

Connection	This header instructs the browser whether to use persistent in HTTP connections or not. A value of **close**instructs the browser not to use persistent HTTP connections and **keep-alive** means using persistent connections.
Content-Disposition	This header lets you request that the browser ask the user to save the response to disk in a file of the given name.
Content-Encoding	This header specifies the way in which the page was encoded during transmission.
Content-Language	This header signifies the language in which the document is written. For example en, en-us, ru, etc.
Content-Length	This header indicates the number of bytes in the response. This information is needed only if the browser is using a persistent (keep-alive) HTTP connection.
Content-Type	This header gives the MIME (Multipurpose Internet Mail Extension) type of the response document.
Expires	This header specifies the time at which the content should be considered out-of-date and thus no longer be cached.
Last-Modified	This header indicates when the document was last changed. The client can then cache the document and supply a date by an **If-Modified-Since** request header in later requests.
Location	This header should be included with all responses that have a status code in the 300s. This notifies the browser of the document address. The browser automatically reconnects to this location and retrieves the new document.
Refresh	This header specifies how soon the browser should ask for an updated page. You can specify time in number of seconds after which a page would be refreshed.
Retry-After	This header can be used in conjunction with a 503 (Service Unavailable) response

	to tell the client how soon it can repeat its request.
Set-Cookie	This header specifies a cookie associated with the page.

Some important methods in response header

setDateHeader(String header, long milliseconds)

This method saves you the trouble of translating a Java date in milliseconds since 1970 (as returned by
System.currentTimeMillis, Date.getTime, or Calendar.getTimeInMillis) into a GMT time string.

void addHeader(String name, String value)
Adds a response header with the given name and value.

setIntHeader(String header, int headerValue)

This method spares you the minor inconvenience of converting an int to a String before inserting it into a header.

void addIntHeader(String name, int value)
Adds a response header with the given name and integer value.

void addCookie(Cookie cookie)

Adds the specified cookie to the response.

setContentType
This method sets the Content-Type header and is used by the majority of servlets

setContentLength
This method sets the Content-Length header, which is useful if the browser supports persistent

sendRedirect
The sendRedirect method sets the Location header as well as setting the status code to 302.

11.Explain about HTTP status code
When a Web server responds to a request from a browser or other Web client, the response typically consists of a status line, some response headers, a blank line, and the document. Here is a minimal example:

HTTP/1.1 200 OK
Content-Type: text/plain

Hello World

The status line consists of the

- HTTP version (HTTP/1.1 in the example above),
- a status code (an integer; 200 in the above example),
- a very short message corresponding to the status code (OK in the example).

200 (OK)
– Everything is fine; document follows.
– Default for servlets.
204 (No Content)
– Browser should keep displaying previous document.
301 (Moved Permanently)
– Requested document permanently moved elsewhere (indicated in Location header).
– Browsers go to new location automatically.
– Browsers are technically supposed to follow 301 and 302 (next page) requests only when the incoming request is GET, but do it for POST with 303. Either way, the Location URL is retrieved with GET.
302 (Found)
– Requested document temporarily moved elsewhere (indicated in Location header).
– Browsers go to new location automatically.
– Servlets should use sendRedirect, not setStatus, when setting this header. See example.
• **401 (Unauthorized)**
– Browser tried to access password-protected page without proper Authorization header.
• **404 (Not Found)**
– No such page. Servlets should use sendError to set this.
– Problem: Internet Explorer and small (< 512 bytes) error pages. IE ignores small error page by default.
– Fun and games: http://www.plinko.net/404/

Setting the status Code
response.setStatus(int statusCode)
– Use a constant for the code, not an explicit int. Constants are in HttpServletResponse
– Names derived from standard message. E.g., SC_OK, SC_NOT_FOUND, etc.

response.sendError(int code, String message)
– Wraps message inside small HTML document

response.sendRedirect(String url)
– Sets status code to 302
– Sets Location response header also
12. What is Session Tracking? Explain different session tracking methods?

- Session tracking is the capability of a server to maintain the current state of a single client's sequential requests.

- Session simply means a particular interval of time.
- Session Tracking is a way to maintain state of a user.
- The HTTP protocol used by Web servers is stateless.
- Each time user requests to the server, server treats the request as the new request.
- So we need to maintain the state of a user to recognize to particular user.
- This type of stateless transaction is not a problem unless you need to know the sequence of actions a client has performed while at your site.
- If Session is not tracking the following problem will come in e-commerce website…..
- When clients at on-line store add item to their shopping cart, how does server know what's already in cart?
- When clients decide to proceed to checkout, how can server determine which previously created cart is theirs?

Why use Session Tracking?

To recognize the user.
To do recommendations.
To advertise

Session Tracking Techniques

There are four techniques used in Session tracking:

- Cookies
- HttpSession
- Hidden Form Field
- URL Rewriting

Cookies – Refer previous answer

Index.html:

```
<form action="FirstServlet" method="post">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java:

```
import java.io.*;
import javax.servlet.*;
```

```
import javax.servlet.http.*;
public class FirstServlet extends HttpServlet {
public void doPost(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
String n=request.getParameter("userName");
out.print("Welcome "+n);
Cookie ck=new Cookie("uname",n);//creating cookie object
response.addCookie(ck);//adding cookie in the response
//creating submit button
out.print("<form action='SecondServlet' method='post'>");
out.print("<input type='submit' value='go'>");
out.print("</form>");
out.close();
}catch(Exception e){System.out.println(e);}
}}
```

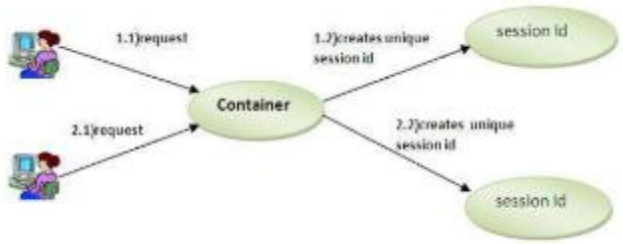
SecondServlet.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SecondServlet extends HttpServlet {
public void doPost(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
Cookie ck[]=request.getCookies();
out.print("Hello "+ck[0].getValue());
out.close();
}catch(Exception e){System.out.println(e);}}
```

2.HttpSession Interface

In such case, container creates a session id for each user.The container uses this id to identify the particular user.An object of HttpSession can be used to perform two tasks:

1. bind objects
2. view and manipulate information about a session, such as the session identifier, creation time, and last accessed time.



How to get the HttpSession object ?

The HttpServletRequest interface provides two methods to get the object of HttpSession:

1. **public HttpSession getSession():**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **public HttpSession getSession(boolean create):**Returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session.

Commonly used methods of HttpSession interface

1. **public String getId():**Returns a string containing the unique identifier value.
2. **public long getCreationTime():**Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
3. **public long getLastAccessedTime():**Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.
4. **public void invalidate():**Invalidates this session then unbinds any objects bound to it.

Example of using HttpSession

In this example, we are setting the attribute in the session scope in one servlet and getting that value from the session scope in another servlet. To set the attribute in the session scope, we have used the setAttribute() method of HttpSession interface and to get the attribute, we have used the getAttribute method.

index.html

```
<form action="servlet1">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;


public class FirstServlet extends HttpServlet {

public void doGet(HttpServletRequest request, HttpServletResponse response){
      try{

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      String n=request.getParameter("userName");
      out.print("Welcome "+n);

      HttpSession session=request.getSession();
      session.setAttribute("uname",n);

      out.print("<a href='servlet2'>visit</a>");

      out.close();

            }catch(Exception e){System.out.println(e);}
   }

}
```

SecondServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SecondServlet extends HttpServlet {

public void doGet(HttpServletRequest request, HttpServletResponse response)
      try{

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      HttpSession session=request.getSession(false);
      String n=(String)session.getAttribute("uname");
      out.print("Hello "+n);
```

```
      out.close();

            }catch(Exception e){System.out.println(e);}
   }


}
```

Hidden Form fields

- Hidden Form fields are added to an HTML form that are not displayed in the client's browser.
- They are sent back to the server when the form that contains them is submitted.

Advantages :

- The advantages of hidden form fields are their ubiquity and support for anonymith
- Hidden fields are supported in all the popular browsers,
- They demand no special server requirements
- They can be used with clients that have not registered or loggin

Disadvantage

- It works only for the sequence of dynamically generated forms
- This technique breaks down immediately with static documents. E-mailed documents, bookmarked documents, and browser shutdowns

Example prg for hidden form field

Index.html:

```
<form action="FirstServlet" method="post">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java:

```
import java.io.*;
```

```
import javax.servlet.*;
import javax.servlet.http.*;


public class FirstServlet extends HttpServlet {
public void doGet(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
String n=request.getParameter("userName");
out.print("Welcome "+n);


//creating submit button
out.print("<form action='SecondServlet'>");
out.print("<input type='hidden' name='userName' value='"+n+"'>");
out.print("<input type='submit' value='go'>");
out.print("</form>");
out.close();
}catch(Exception e){System.out.println(e);}
}}
```

SecondServlet.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SecondServlet extends HttpServlet {
public void doPost(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
 String n=request.getParameter("userName");
out.print("Welcome "+n);
out.close();
}catch(Exception e){System.out.println(e);}}
```

URL Rewriting

- With URL rewriting, every local URL the user might client on is dynamically modified or rewritten, to include extra information.
- The extra info. can be in the form of extra path information, added parameters or some custom, server-specific URL change.

URL?name1=value1&name2=value2

Advantages

- IT will always work whether cookie is disable or not, so it is broser independent
- Extra form submission is of required on each page

Disadvantage:

- It will work only with links
- It can send only textual information

Example prg for URL rewritting

Index.html:

```
<form action="FirstServlet" method="post">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet {
public void doGet(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
String n=request.getParameter("userName");
out.print("Welcome "+n);
out.print(<a href=”servlet2?uname=”+n+”>visit</a>”);
out.close();
}catch(Exception e){System.out.println(e);}
}}
```

SecondServlet.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SecondServlet extends HttpServlet {
public void doPost(HttpServletRequest request, HttpServletResponse response){
try{
response.setContentType("text/html");
PrintWriter out = response.getWriter();
 String n=request.getParameter("userName");
out.print("Welcome "+n);
out.close();
}catch(Exception e){System.out.println(e);}}
```

13. What are the different attributes of page directive?

JSP page directive

The page directive defines attributes that apply to an entire JSP page.

Syntax of JSP page directive

```
<%@ page attribute="value" %>
```

Attributes of JSP page directive

- import
- contentType
- extends
- info
- buffer
- language
- isELIgnored
- isThreadSafe
- autoFlush
- session
- pageEncoding
- errorPage
- isErrorPage

1)import

The import attribute is used to import class,interface or all the members of a package.It is similar to import keyword in java class or interface.

Example of import attribute

```
1. <html>
2. <body>
3.
4. <%@ page import="java.util.Date" %>
5. Today is: <%= new Date() %>
6.
7. </body>
8. </html>
```

2)contentType

The contentType attribute defines the MIME(Multipurpose Internet Mail Extension) type of the HTTP response.The default value is "text/html;charset=ISO-8859-1".

7. Example of contentType attribute

```
1. <html>
2. <body>
3.
4. <%@ page contentType=application/msword %>
5. Today is: <%= new java.util.Date() %>
6.
7. </body>
8. </html>
```

3)extends

The extends attribute defines the parent class that will be inherited by the generated servlet.It is rarely used.

4)info

This attribute simply sets the information of the JSP page which is retrieved later by using getServletInfo() method of Servlet interface.

Example of info attribute

```
1. <html>
2. <body>
3.
4. <%@ page info="composed by Sonoo Jaiswal" %>
```

```
5. Today is: <%= new java.util.Date() %>
6.
7. </body>
8. </html>
```

The web container will create a method getServletInfo() in the resulting servlet.For example:

```
1. public String getServletInfo() {
2.   return "composed by Sonoo Jaiswal";
3. }
```

5)buffer

The buffer attribute sets the buffer size in kilobytes to handle output generated by the JSP page.The default size of the buffer is 8Kb.

Example of buffer attribute

```
1. <html>
2. <body>
3.
4. <%@ page buffer="16kb" %>
5. Today is: <%= new java.util.Date() %>
6.
7. </body>
8. </html>
```

6)language

The language attribute specifies the scripting language used in the JSP page. The default value is "java".

7)isELIgnored

We can ignore the Expression Language (EL) in jsp by the isELIgnored attribute. By default its value is false i.e. Expression Language is enabled by default. We see Expression Language later.

```
1. <%@ page isELIgnored="true" %>//Now EL will be ignored
```

8)isThreadSafe

Servlet and JSP both are multithreaded.If you want to control this behaviour of JSP page, you

can use isThreadSafe attribute of page directive.The value of isThreadSafe value is true.If you make it false, the web container will serialize the multiple requests, i.e. it will wait until the JSP finishes responding to a request before passing another request to it.If you make the value of isThreadSafe attribute like:

<%@ page isThreadSafe="false" %>

The web container in such a case, will generate the servlet as:

```
1. public class SimplePage_jsp extends HttpJspBase
2.   implements SingleThreadModel{
3. .......
4. }
```

9)errorPage

The errorPage attribute is used to define the error page, if exception occurs in the current page, it will be redirected to the error page.

Example of errorPage attribute

```
1. //index.jsp
2. <html>
3. <body>
4.
5. <%@ page errorPage="myerrorpage.jsp" %>
6.
7.  <%= 100/0 %>
8.
9. </body>
10. </html>
```

10)isErrorPage

The isErrorPage attribute is used to declare that the current page is the error page.

1. ***Note: The exception object can only be used in the error page.***

Example of isErrorPage attribute

```
1. //myerrorpage.jsp
2. <html>
3. <body>
4.
5. <%@ page isErrorPage="true" %>
6.
7.  Sorry an exception occured!<br/>
```

8. The exception is: <%= exception %>
9.
10. </body>
11. </html>